

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #8 - Semana 9 – Libro C - Capítulo 5
Punteros y arrays

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 23/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 83–113.

Abstract— *Pointers in C are variables that hold memory addresses of other variables, providing a means to perform certain calculations efficiently. This paper explores the relationship between pointers and arrays, highlighting their integration in the C language, especially within the ANSI C specification. Key concepts such as pointer arithmetic, function arguments, and their application in managing memory allocation are discussed. Additionally, the paper examines the syntax intricacies of pointers, particularly when dealing with multidimensional arrays and function pointers. Overall, it emphasizes the importance of disciplined pointer usage to enhance program clarity and maintainability.*

I. Punteros y arreglos

Un puntero es una variable que contiene la dirección de otra variable. En el lenguaje C, los punteros son ampliamente utilizados debido a que, en algunos casos, son la única forma de realizar ciertos cálculos. Además, tienden a generar un código más compacto y eficiente. Los punteros y los arreglos están estrechamente relacionados, por lo que esta sección explora dicha relación y cómo aprovecharla. Aunque el uso descuidado de punteros puede crear programas difíciles de entender, con disciplina pueden aportar claridad y simplicidad.

En la especificación ANSI C, se formalizaron las reglas sobre el manejo de punteros, estableciendo lo que ya era una buena práctica entre programadores experimentados y lo que los compiladores ya imponían. Además, se introdujo el tipo `void *` (puntero a void) para reemplazar `char *` como el tipo adecuado para punteros genéricos.

II. Punteros y direcciones

La memoria en una máquina típica se organiza como un conjunto de celdas consecutivamente numeradas. Cada celda puede contener distintos tipos de datos, como caracteres (char), enteros cortos (short) o enteros largos (long). Un puntero es un grupo de celdas que pueden almacenar la dirección de otra variable. Por ejemplo, si `c` es una variable de tipo `char` y `p` es un puntero que apunta a `c`, la declaración `p = &c` asigna a `p` la dirección de `c`, diciendo que `p` “apunta a” `c`.

El operador unario `&` obtiene la dirección de una variable en memoria, mientras que el operador `*` (indirección o desreferencia) accede al objeto al que apunta el puntero. Por ejemplo, si `ip` es un puntero a un entero y `x` es un entero, `ip = &x` hace que `ip` apunte a `x`, y `y = *ip` asigna el valor de `x` a `y`. Además, `*ip = 0` asigna el valor 0 a `x`.

Los punteros están restringidos a apuntar a tipos específicos de objetos. Sin embargo, existe una excepción: el puntero genérico `void *`, que puede apuntar a cualquier tipo de dato pero no puede ser desreferenciado directamente.

III. Operadores unarios con punteros

Los operadores unarios `*` y `&` tienen una mayor prioridad que los operadores aritméticos. Por lo tanto, la expresión `y = *ip + 1` primero desreferencia el puntero `ip`, obtiene el valor al que apunta, le suma uno, y luego asigna el resultado a `y`. De manera similar, `*ip += 1` incrementa el valor al que apunta el puntero, lo cual es equivalente a las expresiones `++*ip` y `(*ip)++`. Es importante el uso de paréntesis en `(*ip)++`, ya que sin ellos se incrementaría la dirección del puntero en lugar del valor al que apunta.

Los punteros también pueden ser manipulados directamente. Por ejemplo, si `iq` es otro puntero, la expresión `iq = ip` copia el contenido de `ip` en `iq`, haciendo que ambos apunten al mismo lugar.

IV. Punteros como argumentos de funciones

En C, los argumentos de funciones se pasan por valor, lo que significa que la función llamada no puede modificar las variables originales de la función que la invoca. Para lograr que una función modifique los valores de las variables originales, se pasan punteros como argumentos. Por ejemplo, en lugar de usar `swap(a, b)` para intercambiar dos valores, se debe pasar la dirección de las variables con `swap(&a, &b)`.

Dentro de la función `swap`, los parámetros se declaran como punteros, y los valores se acceden indirectamente. Así, la función puede cambiar los valores originales en la función que la invocó.

V. Punteros y funciones de entrada

Un ejemplo común de uso de punteros en funciones es el caso de `getint`, una función que convierte caracteres de entrada en un valor entero. `getint` devuelve el estado de fin de archivo (EOF) como su valor de retorno, y utiliza un argumento puntero

para almacenar el entero convertido en la función que la llamó. Este mecanismo es similar al utilizado por la función `scanf`.

En el siguiente código, `getint` llena un arreglo con valores enteros obtenidos de la entrada:

```
int n, array[SIZE], getint(int *);
for (n = 0; n < SIZE && getint(&array[n]) != EOF; n++);
```

Es crucial pasar la dirección de `array[n]` a `getint` para que pueda almacenar el número convertido en el arreglo. La función devuelve EOF si se encuentra el fin de archivo, cero si no es un número válido y un valor positivo si el número es válido.

VI. Punteros y arreglos en C

En C, los punteros y los arreglos tienen una relación cercana, tanto que las operaciones realizadas con índices en arreglos pueden ser realizadas también con punteros. Los punteros suelen ser más rápidos, pero son más difíciles de entender para los principiantes.

A. Declaración de arreglos y punteros: Un arreglo de tamaño 10, como `int a[10]`, define un bloque de 10 objetos consecutivos, desde `a[0]` hasta `a[9]`. Se puede acceder a los elementos del arreglo utilizando el índice, como en `a[i]`. Un puntero, como `int *pa`, puede apuntar al primer elemento del arreglo usando `pa = &a[0]`, lo que significa que `pa` contiene la dirección de `a[0]`. Así, se puede acceder al contenido de `a[0]` usando `*pa`.

B. Aritmética de punteros: Si un puntero apunta a un elemento de un arreglo, como `pa[0]`, entonces `pa + 1` apuntará al siguiente elemento, y `*(pa + 1)` accederá al contenido de ese elemento. Este concepto se aplica independientemente del tipo de los elementos del arreglo.

C. Equivalencia entre índices y punteros: En C, el valor de una expresión de tipo arreglo es la dirección de su primer elemento, lo que permite que la asignación `pa = &a[0]` sea equivalente a `pa = a`. Además, la expresión `a[i]` es equivalente a `*(a + i)`, lo que significa que se puede utilizar indistintamente la notación con índices o con punteros.

D. Limitaciones de los nombres de arreglos: Un puntero es una variable que puede ser modificada, mientras que el nombre de un arreglo no lo es. Por ejemplo, `pa = a` es válido, pero `a = pa` no lo es, ya que el nombre del arreglo no puede ser reasignado.

E. Pasar arreglos a funciones: Cuando se pasa un arreglo a una función, lo que realmente se pasa es la dirección de su primer elemento. Por lo tanto, dentro de la función, el arreglo es tratado como un puntero. Las declaraciones `char s[]` y `char *s` son equivalentes como parámetros de una función, ya que ambas indican que se está trabajando con un puntero.

F. Subarreglos: Es posible pasar una parte de un arreglo a una función pasando un puntero que apunte al inicio del subarreglo. Por ejemplo, las expresiones $f(&a[2])$ y $f(a + 2)$ son equivalentes y le pasan a la función la dirección del subarreglo que empieza en $a[2]$.

VII. Aritmética de direcciones

La aritmética de direcciones en C es simple pero poderosa. Operaciones como $p++$ incrementan el puntero p para que apunte al siguiente elemento, y $p += i$ incrementa el puntero para que apunte i elementos más allá.

A. Consistencia en la aritmética de punteros: C es consistente al integrar punteros, arreglos y la aritmética de direcciones. Esto es una fortaleza del lenguaje. Un ejemplo es un rudimentario asignador de memoria que maneja bloques de caracteres. Se puede usar un arreglo, como `allocbuf`, para almacenar los datos, y un puntero, `allocp`, que apunta al próximo bloque libre. La función `alloc(n)` retorna un puntero a un bloque de n caracteres, y `afree(p)` libera la memoria apuntada por p .

B. Uso de punteros en la asignación de memoria: Un puntero puede inicializarse como cualquier otra variable. Por ejemplo, `static char *allocp = allocbuf` define un puntero `allocp` e inicializa su valor para que apunte al inicio de `allocbuf`. Esta asignación podría haberse escrito como `static char *allocp = &allocbuf[0]`.

C. Comparaciones y operaciones con punteros: Es posible comparar punteros, especialmente si apuntan a miembros del mismo arreglo. Por ejemplo, $p < q$ es verdadero si p apunta a un elemento anterior a q en el arreglo. También se pueden sumar y restar punteros: $p + n$ apunta al objeto que está n posiciones más allá de p .

D. Ejemplo con `strlen`: Una implementación de `strlen`, que retorna la longitud de una cadena, utiliza la aritmética de punteros de manera eficiente. El puntero p se inicializa al primer carácter de la cadena, y en cada iteración del bucle se incrementa hasta llegar al carácter nulo `\0`. La diferencia entre los punteros p y s al final del bucle da como resultado la longitud de la cadena.

VIII. Diferencias de tipos de punteros y enteros

Los punteros y los enteros no son intercambiables en C. Sin embargo, el valor 0 es una excepción, ya que puede ser asignado a un puntero o comparado con él. Para mayor claridad, se utiliza la constante `NULL`, que está definida en `<stdio.h>`, para representar un puntero nulo.

IX. Punteros a caracteres y funciones

En C, las cadenas de caracteres son arreglos de caracteres terminados con el carácter nulo (`\0`). Cuando una cadena se pasa como argumento a una función, lo que realmente se pasa es un puntero al primer carácter de esa cadena. También

se pueden asignar a punteros, lo que permite modificar la referencia del puntero, pero no el contenido de la cadena si esta es constante, ya que puede causar comportamiento indefinido.

X. Arreglos de punteros; punteros a punteros

Los punteros pueden almacenarse en arreglos, como en programas que ordenan líneas de texto. Utilizar punteros en lugar de mover las líneas minimiza la sobrecarga de almacenamiento y procesamiento. Un programa para ordenar líneas de texto almacena punteros a las líneas en un arreglo y utiliza funciones para leer, ordenar e imprimir las líneas. Al ordenar, se intercambian los punteros en lugar de mover las líneas mismas.

XI. Arrays multidimensionales en C

C provee arrays multidimensionales, aunque se prefieren arrays de punteros en la práctica. Un ejemplo común es la conversión de fechas entre día del mes y día del año. Esto se logra con funciones que utilizan un array bidimensional para organizar los días de cada mes, separando los años bisiestos de los comunes. Los arrays multidimensionales pueden pasarse a funciones, pero la cantidad de columnas debe estar explícita en la declaración del parámetro.

XII. Inicialización de arrays de punteros

Una función como `month_name(n)`, que devuelve el nombre de un mes, utiliza arrays de punteros a cadenas de caracteres. El array se inicializa con los nombres de los meses y devuelve el nombre correspondiente según el número del mes.

XIII. Punteros vs arrays multidimensionales

Un array bidimensional y un array de punteros tienen diferencias importantes. Un array bidimensional ocupa un bloque contiguo de memoria, mientras que un array de punteros solo almacena punteros a otras ubicaciones, lo que permite flexibilidad en el tamaño de las filas. Los arrays de punteros son útiles cuando se trabaja con datos de longitud variable, como cadenas de caracteres.

XIV. Argumentos de línea de comandos

En C, los argumentos se pueden pasar a un programa a través de la línea de comandos. Estos se reciben en `main` mediante dos parámetros: `argc`, que cuenta los argumentos, y `argv`, un array de punteros a cadenas que contienen los argumentos. Por ejemplo, `argv[0]` contiene el nombre del programa, y `argv[1]` en adelante contienen los argumentos opcionales.

XV. Declaraciones complicadas

El lenguaje C es criticado por la sintaxis de sus declaraciones, en especial cuando involucran punteros a funciones. El objetivo de la sintaxis es hacer que la declaración y el uso coincidan. Aunque funciona bien en casos simples, puede ser

confuso en situaciones más complejas, debido a que las declaraciones no se leen de izquierda a derecha y al uso excesivo de paréntesis.

Diferencias entre declaraciones

Un ejemplo común de confusión es la diferencia entre las siguientes declaraciones:

- `int *f();` que indica una función que retorna un puntero a un entero.
- `int (*pf)();` que representa un puntero a una función que retorna un entero.

La confusión surge porque el asterisco (*) es un operador prefijo que tiene menor precedencia que los paréntesis. Es necesario usar paréntesis para garantizar la asociación adecuada.

Sugerencias para sintetizar declaraciones complejas

Aunque las declaraciones verdaderamente complicadas rara vez aparecen en la práctica, es importante saber cómo entenderlas y crearlas si es necesario. Una buena manera de sintetizarlas es usar typedef, que se explica en la Sección 6.7. Otra alternativa es utilizar programas que conviertan una declaración válida en C en una descripción en palabras y viceversa.

Ejemplos de conversión de declaraciones

El programa dcl convierte declaraciones en C en descripciones de palabras, como en estos ejemplos:

- `char **argv` se describe como "puntero a char".
- `int (*daytab)[13]` se describe como "puntero a un array[13] de int".
- `void *comp()` se describe como "función que retorna un puntero a void".

Gramática simplificada de declaradores

La gramática que especifica un declarador en C es la siguiente:

- `dcl`: opcionales * seguidos de `direct-dcl`.
- `direct-dcl`: puede ser un nombre, un `dcl` entre paréntesis, seguido de paréntesis para funciones, o de corchetes para arrays.

Esta gramática se puede usar para analizar funciones. Por ejemplo, en la declaración `(*pfa[])()`, `pfa` se identifica como un nombre, luego `pfa[]` es un `direct-dcl`, y finalmente `(*pfa[])()` se analiza como un `dcl` completo.

Estructura del programa dcl: El programa dcl utiliza dos funciones principales: `dcl` y `dirdcl`. Estas funciones analizan una declaración siguiendo la gramática mencionada. Dado que la gramática es recursiva, las funciones se llaman entre sí de manera recursiva para reconocer las partes de la declaración.

```

void dcl(void) {
    int ns;
    for (ns = 0; gettoken() == '*'; ) ns++;
    dirdcl();
    while (ns-- > 0) strcat(out, " puntero a");
}

```

La función `dirdcl` se encarga de analizar el declarador directo.

```

void dirdcl(void) {
    int type;
    if (tokentype == '(') {
        dcl();
        if (tokentype != ')') printf("error: falta ')\n");
    } else if (tokentype == NAME) {
        strcpy(name, token);
    } else {
        printf("error: se esperaba un nombre o (dcl)\n");
    }
}

```

Restricciones del programa dcl: El programa tiene varias limitaciones. Solo maneja tipos de datos simples como `char` o `int`, no maneja tipos de argumentos en funciones ni calificadores como `const`. Además, los espacios en blanco incorrectos pueden confundir el programa, y no cuenta con un manejo robusto de errores.

Conversión inversa con undcl: El programa `undcl` realiza la conversión inversa, es decir, convierte descripciones de palabras en declaraciones en C. Por ejemplo, la descripción "x es una función que retorna un puntero a un array de punteros a funciones que retornan char" se convierte en la declaración `char ((*x())[])()`.

Este programa reutiliza la función `gettoken` del programa `dcl` y utiliza las mismas variables externas.

```

main() {
    while (gettoken() != EOF) {
        strcpy(out, token);
        while ((type = gettoken()) != '\n') {
            if (type == PARENS || type == BRACKETS) strcat(out, token);
            else if (type == '*') sprintf(temp, "(*%s)", out);
            else if (type == NAME) sprintf(temp, "%s %s", token, out);
        }
    }
    return 0;
}

```

Este proceso de conversión es más sencillo que el análisis de declaraciones, pero aún puede generar redundancias en los paréntesis.